

Algoritmos y Estructuras de Datos – Segundo Semestre 2026

Problem Sets UT0 - Transición a JAVA – Ejercicios Obligatorios.

Ejercicio 1. Primer programa Java, variables e inicialización

- Crear una clase llamada PruebaAtributos dentro de un paquete definido por el equipo.
- Agregar atributos de instancia de distintos tipos (int, boolean, double, char y String) y mostrar sus valores por defecto.
- Intentar declarar nombres válidos e inválidos para variables y registrar qué errores genera el compilador.
- Repetir la experiencia con variables locales sin inicializar y explicar la diferencia con los atributos.
- Agregar comentarios de una línea y de varias líneas, y documentar en el README cómo compilar y ejecutar el programa desde línea de comandos.

Entrega mínima

- Código fuente funcionando.
- Breve explicación escrita de JVM, JDK y JRE.
- Tabla pequeña con ejemplos de tipos primitivos y de referencia usados en el programa.

Bibliografía

- Oracle Java Tutorial - Variables.
- Handbook de Java, secciones 1 a 4.

Ejercicio 2. Operadores, expresiones y conversiones básicas

- Revisar el programa ArithmeticDemo e indicar en qué líneas conviene reemplazar asignaciones simples por asignaciones compuestas.
- Explicar con detalle qué ocurre en la instrucción: `int a = 5; int i = 3; a += ++i;`
- Agregar una variante que reciba dos valores por línea de comandos, los convierta a números y realice operaciones básicas.

Entrega mínima

- Programa comentado.
- Explicación escrita del orden de evaluación de al menos tres expresiones.
- Pequeña tabla con conversiones usadas.

```
class ArithmeticDemo {
    public static void main(String[] args) {
        int result = 1 + 2;
        result = result - 1;
        result = result * 2;
        result = result / 2;
        result = result + 8;
        result = result % 7;
    }
}
```

Bibliografía

- Oracle Java Tutorial - Operators.
- Oracle Java Tutorial - Expressions.
- Oracle Java Tutorial - Converting Between Numbers and Strings.

- Handbook de Java, secciones 5 y 20.

Ejercicio 3. Contador incremental y control de flujo

- Crear una clase Contador con los atributos MAX_CONT, incremento y contador; declarar MAX_CONT como final.
- Implementar el conteo incremental con while, luego con do-while y finalmente con for, verificando que el comportamiento sea equivalente.
- Agregar un menú simple con switch que permita elegir qué variante de conteo ejecutar.
- Incluir una demostración breve de la diferencia entre un atributo static y uno de instancia dentro de la misma clase o en una clase auxiliar.
- Explicar en no más de diez líneas en qué casos usarías while, do-while y for.

Entrega mínima

- Código con las tres implementaciones.
- Breve comparación escrita entre las estructuras de control utilizadas.

Bibliografía

- Handbook de Java, secciones 6, 17 y 18.

Ejercicio 4. Factorial, primalidad y sumas condicionales

- Implementar un método factorial(int num) usando únicamente bucles for.
- Contemplar y documentar casos borde: 0, 1, negativos y valores demasiado grandes para el tipo elegido.
- Implementar un programa que determine si un número es primo.
- Si el número es primo, calcular la suma de los pares entre 0 y ese valor; si no lo es, calcular la suma de los impares.
- Para la segunda parte utilizar únicamente while o do-while.
- Separar la solución en métodos bien nombrados y reutilizables.

Entrega mínima

- Código fuente.
- Juego mínimo de pruebas manuales con entradas y salidas esperadas.
- Explicación de al menos dos decisiones de diseño.

Base o pista de arranque

```
public class UtilMath {
    public static int factorial(int num) {
        return 0; // TODO
    }

    public static boolean isPrime(long n) {
        return false; // TODO
    }
}
```

Bibliografía

- Handbook de Java, secciones 6 y 7.

Ejercicio 5. Marcapasos: tipos de datos, objetos y memoria

- Definir una clase Marcapasos con los atributos idDispositivo, codigoFabricante, latidosPorMinuto, nivelBateria, eligiendo tipos que minimicen memoria sin perder significado.
- Agregar al menos un constructor, getters y setters donde corresponda, y justificar por qué algunos atributos podrían no tener setter.
- Incorporar un contador static de instancias creadas y un identificador constante final del dispositivo.
- Implementar toString para mostrar el estado del objeto en forma legible.
- Implementar equals y hashCode tomando como identidad el código del fabricante y el identificador del dispositivo.
- Calcular la memoria consumida considerando solamente las variables declaradas y explicitar qué simplificaciones se están haciendo.

Entrega mínima

- Clase completa.
- Documento breve con el cálculo de memoria.
- Salida de ejemplo usando toString.

Extensión — Jerarquía de dispositivos médicos

La clínica empieza a manejar más de un tipo de dispositivo implantable, y pide generalizar el diseño anterior.

Definir una clase abstracta DispositivoMedico con los atributos comunes a todo dispositivo implantable. Modificar Marcapasos para que extienda de DispositivoMedico. Crear una segunda clase, Desfibrilador, con sus propios atributos (por ejemplo, nivel de descarga y cantidad de descargas aplicadas), que también extienda de DispositivoMedico.

El contador de instancias ahora debe mostrar la cantidad total de dispositivos médicos creados, sin importar el subtipo: justificar la decisión de dónde ubicarlo.

Definir equals y hashCode en DispositivoMedico, usando idDispositivo y codigoFabricante como identidad. Considerar si un Marcapasos y un Desfibrilador con el mismo idDispositivo y codigoFabricante deberían considerarse el mismo dispositivo.

Entrega mínima (extensión)

- Clases completas (DispositivoMedico y las dos subclases).
- Documento breve con el cálculo de memoria y la justificación sobre equals.
- Salida de ejemplo instanciando un Marcapasos y un Desfibrilador, mostrando el contador compartido.

Bibliografía

- Referencia general: Oracle Java Tutorial.
- Handbook de Java, secciones 4, 10, 11, 12, 17, 18, 27 y 28.

Ejercicio 6. Métodos reutilizables, sobrecarga y arreglos numéricos

- Crear un programa Multsuma con un método que calcule $a * b + c$.
- Agregar una versión sobrecargada de multsuma que trabaje con int y otra con double.
- Implementar la multiplicación de dos vectores o arreglos de enteros, validando previamente si la operación es posible según la interpretación elegida.
- Explicar en lenguaje natural la estrategia usada para validar y calcular.
- Separar entrada, cálculo y salida en métodos distintos.

Entrega mínima

- Clase o clases necesarias.
- Explicación breve de la sobrecarga implementada.
- Ejemplos de ejecución con arreglos válidos e inválidos.

Base o pista de arranque

```
public class Multsuma {  
    public static double multsuma(double a, double b, double c) {  
        return a * b + c;  
    }  
}
```

Bibliografía

- Handbook de Java, secciones 7, 8 y 25.

Ejercicio 7. Strings I: análisis, búsquedas y transformaciones

- Tomar el ejemplo de palíndromo del material y probarlo con frases que incluyan mayúsculas, minúsculas y signos de puntuación.
- Explicar por qué String es inmutable y cómo influye eso en la solución.
- Escribir ejemplos simples que usen substring, split, subSequence, trim, toLowerCase, toUpperCase, indexOf, lastIndexOf, contains, replace, replaceAll y replaceFirst.
- Agregar dos microconsultas basadas en la cadena hannah y en una operación substring, indicando longitud, carácter por índice y subcadena obtenida.
- Mostrar la salida por consola de forma clara y etiquetada.

Entrega mínima

- Programa demostrativo.
- Breve informe con observaciones sobre palíndromos y puntuación.

Base o pista de arranque

```
String hannah = "Did Hannah see bees? Hannah did.";  
String frase = "Anita lava la tina";
```

Bibliografía

- Oracle Java Tutorial - Strings.
- Oracle Java Tutorial - Manipulating Strings.

- Handbook de Java, seccion 19.

Ejercicio 8. Strings II: StringBuilder, comparación y pool de Strings

- Explicar qué es StringBuilder y para qué sirve.
- Construir ejemplos pequeños que usen append, insert, delete, deleteCharAt, reverse, setLength y ensureCapacity.
- Indicar la capacidad inicial de un StringBuilder creado a partir de una cadena literal y verificarla programáticamente.
- Analizar paso a paso cómo cambia un StringBuilder luego de varias operaciones encadenadas.
- Explicar por qué comparar String con == puede producir errores lógicos, corregir el caso del bucle infinito y relacionarlo con equals, intern() y el string pool.

Entrega mínima

- Programa demostrativo.
- Explicación escrita de la diferencia entre == y equals para String.

Base o pista de arranque

```
StringBuilder sb = new StringBuilder("Able was I ere I saw Elba.");
String s = "1";
```

Bibliografía

- Oracle Java Tutorial - String Buffers.
- Oracle Java Tutorial - Strings.
- Handbook de Java, seccion 19.

Ejercicio 9. ContadorPalabras orientado a objetos

- Partir de la clase ContadorPalabras del repartido y encapsular correctamente la lógica de conteo.
- Agregar constructores y, si lo consideras útil, una clase ResultadoAnálisis para devolver más de un dato.
- Definir una abstracción razonable (por ejemplo una clase abstracta AnalizadorTexto o una interface ProcesadorTexto) y hacer que ContadorPalabras la utilice o implemente.
- Crear al menos una segunda implementación o subclase para demostrar herencia, polimorfismo y sobreescritura.
- Agregar una sobrecarga de contarPalabras para trabajar tanto con String como con un arreglo o colección de líneas.
- Implementar toString en la clase de resultado y documentar con un ejemplo la diferencia entre sobrecarga y sobreescritura.
- En el README del ejercicio incluir un glosario breve con: clase, objeto, atributo, método, instancia, herencia, polimorfismo, encapsulamiento y abstracción.

Entrega mínima

- Modelo OO funcionando.
- Diagrama simple o explicación de las clases.
- README con glosario y decisiones de diseño.

Base o pista de arranque

```
interface ProcesadorTexto {
    int contarPalabras(String texto);
}
```

```
class ContadorPalabras implements ProcesadorTexto {
    @Override
    public int contarPalabras(String texto) {
        return 0; // TODO
    }
}
```

Bibliografía

- Handbook de Java, secciones 9 a 16, 26, 28, 29, 30 y 31.

Ejercicio 10. Archivos, arreglos y colecciones sobre texto

- Extender ContadorPalabras con un método que obtenga líneas desde un archivo y otro que calcule la cantidad de palabras sobre esas líneas.
- Agregar una operación que reciba dos arreglos de palabras y devuelva las palabras presentes en ambos.
- Resolver la intersección primero con arreglos y luego con una colección de Java, explicando ventajas y desventajas.
- Usar imports y paquetes de forma ordenada.
- Manejar errores básicos de lectura sin romper la ejecución del programa.

Entrega mínima

- Código fuente.
- Una pequeña comparación entre la solución con arreglos y la solución con colecciones.

Base o pista de arranque

```
class ContadorPalabras {
    public String[] obtenerLineas(String archivo) {
        return new String[0];
    }

    public String[] palabrasComunes(String[] a, String[] b) {
        return new String[0];
    }
}
```

Bibliografía

- Material de manejo básico de lectura/escritura de archivos.
- Handbook de Java, secciones 8, 21, 22 y 23.

Ejercicio 11. Entrada de datos por archivo y por teclado

- Parte A: leer un archivo de entrada y mostrar por consola el entero, el real, la cadena, la suma y la división entera con resto.
- Parte B: leer desde stdin el radio de una circunferencia y devolver su área y perímetro.
- Usar Scanner al menos en una de las partes.
- Controlar errores de formato de entrada de manera razonable.
- Separar la solución en métodos estáticos dentro de una clase Principal.

Entrega mínima

- Código funcionando.
- Ejemplo de archivo de entrada y captura de una ejecución desde teclado.

Base o pista de arranque

```
public class Principal {  
    public static void leerEntradaArchivo(String rutaArchivo) {  
        // TODO  
    }  
  
    public static void leerEntradaStdin() {  
        // TODO  
    }  
}
```

Bibliografía

- Handbook de Java, secciones 22, 23 y 24.

Ejercicio 12. Depuración y corrección de errores frecuentes

- Detectar y corregir los errores planteados en los fragmentos del repartido original.
- Explicar cómo identificaste cada problema y qué mensaje o síntoma te permitió llegar a la causa.
- Agregar para cada caso una versión mínima que no falle y otra que falle de manera controlada.
- Indicar qué buenas prácticas habrían evitado el error desde el principio.

Entrega mínima

- Archivo con correcciones.
- Breve bitácora de depuración.

Base o pista de arranque

Registrar para cada fragmento: problema observado, causa probable, corrección aplicada y evidencia de que ahora funciona.

Bibliografía

- Handbook de Java, secciones 22 y 29.

Ejercicio 13. Tipos enumerados aplicados al análisis de texto

- Escribir un ejemplo sencillo que use un enum y recorra sus valores con values().
- Replantear una parte del contador de vocales y consonantes usando un tipo enumerado.
- Explicar en qué mejora la legibilidad o mantenibilidad de la solución frente a una alternativa sin enum.

Entrega mínima

- Ejemplo de enum funcionando.
- Breve explicación escrita.

Base o pista de arranque

```
enum TipoCaracter {  
    VOCAL,  
    CONSONANTE,  
    DIGITO,  
    OTRO  
}
```

Bibliografía

- Oracle Java Tutorial - Enum Types.

- Handbook de Java, sección 9 y refuerzo de buenas prácticas.

Ejercicio 14. Proyecto integrador T9 con pruebas JUnit 5

- Implementar la transformación a T9 leyendo texto desde un archivo y escribiendo la salida en otro archivo.
- Agregar una variante que invierta el texto antes de convertirlo.
- Organizar el proyecto con estructura `src/main/java` y `src/test/java`.
- Escribir al menos: un test básico, un test parametrizado o repetido, un test que verifique excepción y un test con timeout.
- Usar anotaciones de ciclo de vida cuando aporten claridad, y nombrar los tests siguiendo una convención consistente.
- Ejecutar la suite con Maven y dejar evidencia del resultado.

Entrega mínima

- Código del transformador T9.
- Paquete de tests JUnit 5.
- README breve con instrucciones para ejecutar `mvn test` y checklist final.

Base o pista de arranque

```
mvn test

class TransformadorT9Test {
    // escribir aquí los tests principales
}
```

Bibliografía

- JUnit Handbook Sencillo - secciones 1, 2, 3, 4, 5, 8, 9, 10 y 11.